

Como funciona Jaimito

Flujo operativo del bot de Telegram desplegado en Railway con Docker y OpenAI API

Resumen ejecutivo

Jaimito es un bot de Telegram para el curso de IA aplicada. Su función es ayudar al alumnado a preparar una web sencilla de proyecto final. El bot no vive en el ordenador del profesor: se ejecuta en Railway, dentro de una imagen Docker, y llama a OpenAI cuando necesita generar una respuesta personalizada.

Idea clave: Telegram es el canal, Railway es el servidor que mantiene el bot encendido, Docker es la caja reproducible que contiene el bot, y OpenAI es el proveedor de IA que genera respuestas cuando hace falta.

Flujo general

```
Alumno en Telegram
-> Telegram Bot API
-> Jaimito en Railway
-> Contenedor Docker
-> Código Python del bot
-> OpenAI API, si hace falta
-> Respuesta en Telegram
```

Lectura rápida del flujo

- El alumno escribe un comando en Telegram, por ejemplo /pregunta o /web.
- Telegram entrega el mensaje al bot mediante polling.
- Railway mantiene el proceso de Jaimito encendido 24/7.
- El proceso corre dentro de una imagen Docker publicada en GitHub Container Registry.
- Si la respuesta requiere IA, Jaimito llama a OpenAI.
- Jaimito limpia la respuesta y la devuelve al grupo de Telegram.

Componentes principales

Componente	Que hace	Equivalente sencillo
Telegram	Canal donde escriben los alumnos y donde Jaimito responde.	La sala de chat del curso.
BotFather / token	Identifica a Jaimito ante Telegram mediante TELEGRAM_BOT_TOKEN.	La llave de identidad del bot.
Polling	Jaimito pregunta continuamente a Telegram si hay mensajes nuevos.	Estar mirando la bandeja de entrada.
Railway	Servidor cloud que mantiene Jaimito encendido aunque el PC local este apagado.	Un servidor gestionado 24/7.
Docker	Empaqueta Python, dependencias y código para que el bot arranque igual en cualquier entorno.	Una caja con todo preparado.
GHCR	Registro donde se guarda la imagen Docker ghcr.io/josegodev/jaimitobot:latest.	Almacén de imágenes Docker.

OpenAI API	Genera respuestas cuando el usuario usa /pregunta, /web o una mencion.	Motor de IA externo.
Logs	Permiten depurar errores, latencias y eventos internos.	Cuaderno de incidencias tecnico.

1. Entrada desde Telegram

El usuario escribe en Telegram. Jaimito no responde a todos los mensajes normales: responde a comandos o menciones para reducir ruido y coste.

Ejemplos de uso

```
/start
/ayuda
/pregunta como creo una web?
/web quiero hacer una web sobre una tienda con IA
/plantilla_web
/rubrica
```

Los mensajes normales como 'hola', 'ok' o conversaciones del grupo pueden ignorarse. Esto evita que cada comentario consuma tokens de OpenAI.

2. Polling: como recibe mensajes

Jaimito usa polling. Esto significa que el proceso del bot pregunta a Telegram de forma continua si hay mensajes nuevos. No se usa webhook ni servidor HTTP publico.

```
Jaimito -> Telegram: hay mensajes nuevos?
Telegram -> Jaimito: si, llego /pregunta ...
Jaimito procesa el mensaje
Jaimito responde en Telegram
```

Consecuencia operacional: no hace falta exponer un dominio, abrir un puerto HTTP, crear FastAPI ni definir un healthcheck web. El bot solo necesita mantenerse ejecutando el proceso Python.

3. Railway: servidor 24/7

Railway aloja el proceso de Jaimito. Si el ordenador local se apaga, el bot sigue funcionando porque vive en Railway.

Railway aporta:

- ejecucion continua del contenedor
- variables de entorno privadas
- logs de ejecucion
- reinicio si el proceso falla
- despliegue desde una imagen Docker publicada

En este proyecto no hace falta networking publico en Railway porque Telegram se comunica con el bot a traves del polling saliente. El contenedor no necesita recibir peticiones HTTP desde internet.

4. Docker: caja reproducible

Docker define un entorno cerrado con Python, dependencias y el codigo de Jaimito. Esto reduce diferencias entre la maquina local, Railway u otro proveedor.

Contenido conceptual de la imagen Docker:

```
Python 3.12
requirements.txt
telegram_web_assistant/
CMD: python telegram_web_assistant/jaimito_bot.py
```

La imagen Docker se publica en GHCR con un nombre como:

```
ghcr.io/josegodev/jaimitobot:latest
```

Railway descarga esa imagen y crea un contenedor. El contenedor arranca el comando definido en el Dockerfile.

5. OpenAI API: cuando se usa IA

OpenAI solo interviene cuando Jaimito necesita generar una respuesta flexible. No todos los comandos llaman a OpenAI.

Tipo	Comandos	Consumo OpenAI
Respuesta local	/start, /ayuda, /plantilla_web, /rubrica, /estado	No
Respuesta generada	/pregunta, /web, menciones al bot	Si

Jaimito envia a OpenAI un mensaje con instrucciones educativas: responder claro, ayudar a alumnos no tecnicos y orientar la respuesta a la web del proyecto final.

6. Respuesta al alumno

Despues de recibir la respuesta de OpenAI, Jaimito limpia el texto y lo manda de vuelta a Telegram. La respuesta debe ser comprensible para el alumnado y no mostrar informacion interna.

Regla importante:

trace_id en logs internos: SI
trace_id visible en Telegram: NO

El trace_id sirve para depurar, pero no aporta valor al alumno y puede confundir. Por eso se mantiene solo en Railway logs.

7. Variables de entorno

Las claves y configuraciones sensibles no se guardan en el codigo. Se guardan como variables privadas en Railway.

Variable	Funcion
TELEGRAM_BOT_TOKEN	Token del bot creado con BotFather.
OPENAI_API_KEY	Clave para llamar a la API de OpenAI.
OPENAI_MODEL	Modelo usado, por ejemplo gpt-4o-mini.
DEFAULT_TEMPERATURE	Controla creatividad/variacion de respuestas.
TELEGRAM_ALLOWED_CHAT_IDS	Limita el bot a grupos autorizados.

Estas variables no deben aparecer en GitHub, Dockerfile, README publico, capturas ni mensajes de Telegram.

8. Control de acceso y coste

El bot esta pensado para un grupo de curso. Para evitar uso indebido, se recomienda configurar TELEGRAM_ALLOWED_CHAT_IDS con el id del grupo autorizado.

Medidas de control:

- responder solo a comandos o menciones
- ignorar mensajes normales del grupo
- usar un modelo economico cuando sea suficiente

- mantener logs para depurar fallos
- no exponer claves en código ni en imagen Docker

9. Flujo detallado de una pregunta

Ejemplo: el alumno escribe /pregunta como creo una web?

1. El alumno escribe /pregunta como creo una web?
2. Telegram registra el mensaje.
3. Jaimito, en Railway, lo recibe por polling.
4. El codigo Python identifica el comando /pregunta.
5. Jaimito prepara el prompt educativo.
6. Jaimito llama a OpenAI con OPENAI_API_KEY.
7. OpenAI devuelve una respuesta.
8. Jaimito limpia la respuesta.
9. Jaimito envia la respuesta a Telegram.
10. Railway conserva logs internos.

10. Flujo de despliegue con Docker

Cuando se modifica el codigo del bot, el flujo de despliegue con imagen Docker es:

```
Cambio en codigo
-> git commit
-> git push a GitHub
-> docker build
-> docker push a GHCR
-> Railway redeploy
-> Railway descarga la imagen
-> Jaimito arranca dentro del contenedor
```

Riesgo: si Railway usa la etiqueta latest, es comodo pero menos trazable. Para operacion mas seria conviene usar etiquetas versionadas como v1, v2, v3.

11. Fallos tipicos y diagnostico

Sintoma	Causa probable	Revision
El bot no responde	Servicio parado, token incorrecto o conflicto de polling.	Railway logs, TELEGRAM_BOT_TOKEN, procesos duplicados.
Conflict getUpdates	Dos procesos usando el mismo token.	Parar bot local o servicio antiguo.
Railway no descarga imagen	Imagen GHCR privada o credenciales incorrectas.	Hacer paquete publico o configurar registry credentials.
/pregunta falla pero /start funciona	Problema con OPENAI_API_KEY, modelo o credito.	Variables Railway y logs.
Aparece trace_id en Telegram	Respuesta visible contaminada con dato interno.	clean_user_message y handlers de reply_text.

12. Buenas practicas actuales

- Mantener una sola instancia activa del bot por token.
- No activar dominio publico ni healthcheck HTTP si el bot usa polling.
- Guardar secretos solo en Railway Variables.
- Mantener Dockerfile y requirements.txt simples.
- Usar logs internos para depurar, pero mensajes limpios para alumnos.

- Preferir respuestas claras, pedagogicas y orientadas al proyecto final.

Conclusion

Jaimito es un asistente operativo para el curso: Telegram es la interfaz, Railway mantiene el proceso encendido, Docker garantiza un entorno reproducible, GHCR almacena la imagen y OpenAI genera respuestas cuando hace falta. La arquitectura es simple, suficiente para un curso y facil de mover a otro proveedor si fuera necesario.

Resumen en una frase

Jaimito escucha comandos en Telegram, corre 24/7 en Railway dentro de Docker, consulta OpenAI cuando necesita IA y responde al alumnado con ayuda practica para crear la web del proyecto final.